

Sistem de analiză a conținutului unei cărți

Mihai Bălan & Bianca Elena Olaru



Originea ideii

Ideea acestui proiect a luat naștere din dorința de a elimina **bariera** dintre textul literar nestructurat și analiza de date obiectivă,

Am pornit de la premisa că volumele mari de text în format PDF ascund **tipare gramaticale și relaționale** complexe care sunt greu de mapat manual, astfel că am propus utilizarea agenților de inteligență artificială pentru a extrage automat „scheletul” unei opere.

Prin acțiuni precum procesarea verbelor pentru a măsura dinamica acțiunii și analizarea dialogurilor pentru a genera grafuri de relații între personaje, sistemul **transformă o carte statică într-o bază de date interactivă**, oferind o perspectivă tehnică și riguroasă asupra structurii narative.

Cum funcționează?

Componentele specifice + tehnologii utilizate:

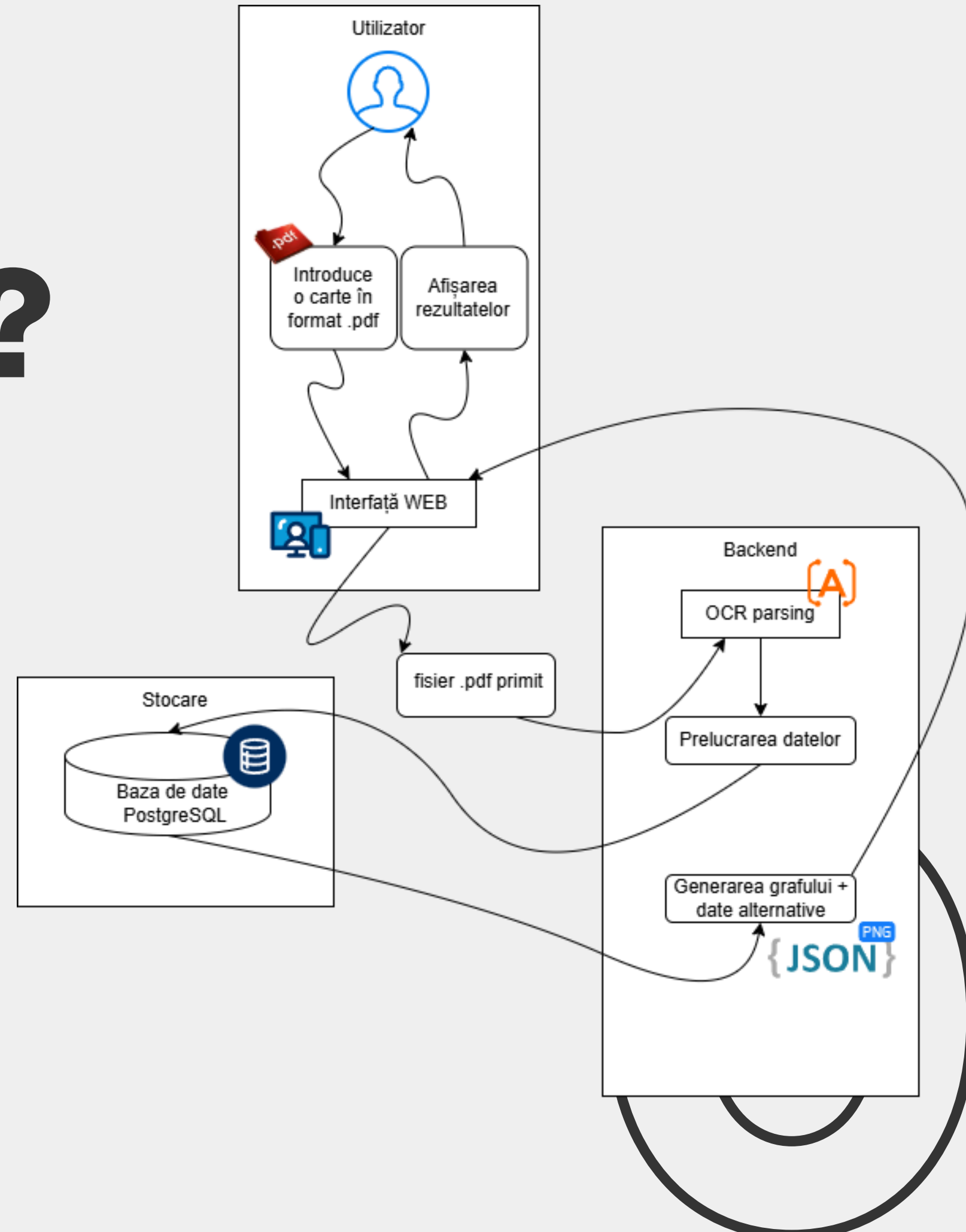
01. Frontend:
Bootstrap + Vis.js (Cytoscape.js), SPA approach

02. Backend:
Django, Python OCR + LLM-uri

03. Baza de date:
PostgreSQL

04. CI/CD:
Github Actions

05. Limbaj utilizat:
Python 3.12



04

■ 01

pdf2image

- fișier .pdf

■ 03

**Tesseract
Pytesseract
+regex**

- OCR parsing

■ 02

openCV

- imagine

■ 04

.json

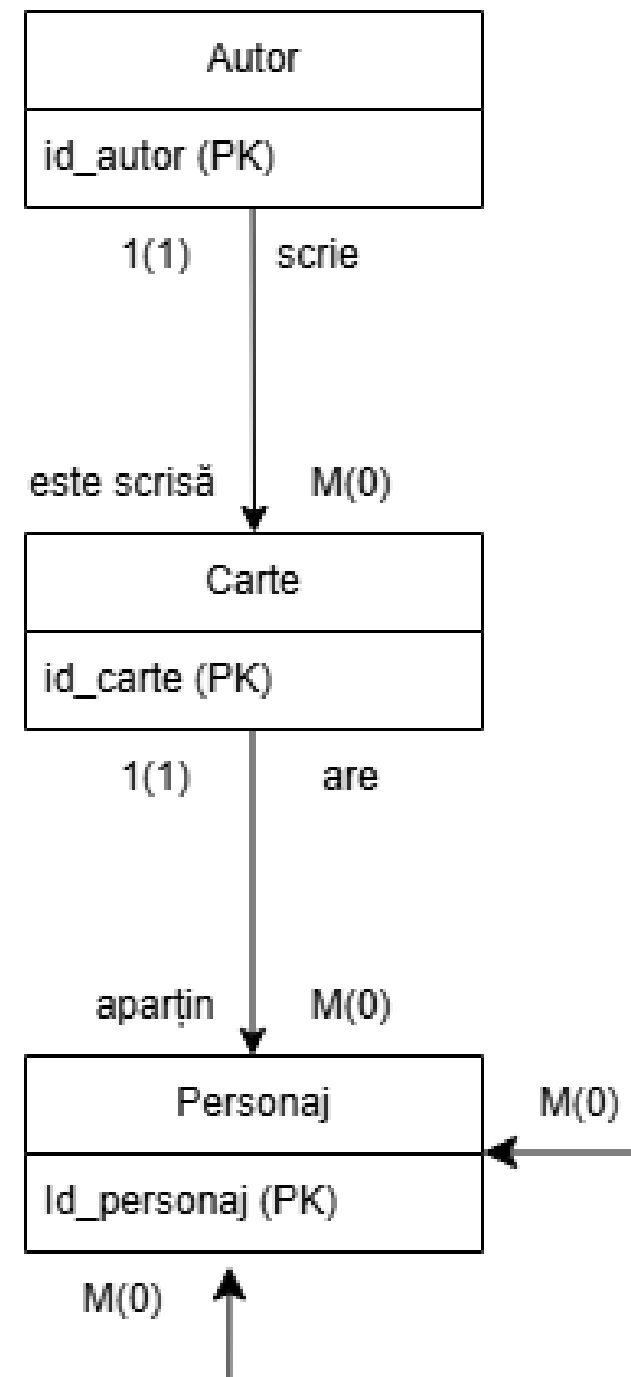
- fișier text

Pipelineul de date

parsarea documentului inițial

Structura bazei de date

Diagrama ER



Relația reflexivă **many to many** pe tabelul *Personaj* se referă la faptul că personajele dialoghează între ele.

Apare tabelul asociativ *Relație* cu foreign key compus și constraint check-uri cu privire la unicitatea conexiunilor care se formează între (graful dialogurilor pentru analiza finală trebuie să fie neorientat, ponderat și nu neapărat conex).

Design patterns

Șabloane pe care le folosim pentru probleme recurente de proiectare software, asigurând un cod modular, scalabil și ușor de întreținut.

01.

Singleton

Garantarea unei conexiuni unice și sigure la baza de date pentru stocarea entităților extrase din carte.

02.

MVC

Separarea logicii de procesare text (Model) de interfața utilizator (View) prin intermediul unui Controller, facilitând mentenanța.

03.

Observer

Actualizarea în timp real a componentelor UI (bara de progres, dashboard) pe măsură ce parser-ul analizează paginile documentului.

Unit Tests

Validarea logicii izolate (ex: algoritmi de curățare text, procesare Regex, calculul scorurilor de relație).

Integration Tests

Verificarea comunicării între componente (ex: flow-ul de date între Parser-ul de PDF și baza de date PostgreSQL).

Functional (E2E) Tests:

Simularea unui flux complet de la upload până la generarea raportului final de analiză.

Testare automată

+ automatizare CI/CD:
validare la fiecare push
deployment pipeline

pytest-django

Gestionarea automată a bazelor de date de test și testarea componentelor web.

pytest-mock

Izolarea aplicației de servicii externe (ex: simularea răspunsurilor de la API-uri de LLM a.î. să reducem costul și timpul de execuție).

pytest-cov

Monitorizarea gradului de acoperire a codului (Code Coverage), vizând un prag de stabilitate de peste 80%.



Observatii finale și pașii următori

Arhitectura actuală a aplicației permite **procesarea/parsarea documentelor** PDF, realizând segmentarea automată a textului pe pagini și unități logice (paragrafe mari) pentru a menține contextul narativ.

În etapa imediat următoare, roadmap-ul proiectului vizează integrarea unei infrastructuri de **LLM-uri**, coordonați într-un mediu containerizat prin Docker, care vor orchestra extragerea metadatelor lingvistice și actualizarea asincronă a bazei de date.

Acești agenți vor avea rolul de a popula structurile relaționale cu entități și perspective deduse, întregul ecosistem fiind ulterior expus printr-o **interfață** de utilizator intuitivă ce va permite vizualizarea în timp real a hărții relațiilor dintre personaje și a dinamicii acțiunii.